

Midterm Report

Title: Dynamic Batch-Size Throttling for GPU di/dt Noise Mitigation

Students: Nikhil Reddy Mummadi, Jakub Mroczek

Repository: <https://github.com/Jakub-Mroczek/SlewGuard>

1. Refined project scope

The core objective remains to build a software closed-loop controller that dynamically throttles GEMM batch size to limit simulated voltage droop (V_{min} violations) driven by GPU power-delivery (PDN) dynamics and di/dt-style transients. The following refinements reflect implementation experience and cluster constraints:

- **Hardware target:** Purdue Scholar cluster, Tesla V100-PCIE-16GB (16 GB VRAM, 250 W TDP), CUDA 12.1, sm_70.
- **Python–C++ bridge:** Implemented with ctypes. The host loads libgemm.so, declares run_gemm, and launches with configurable M, N, K (with M as the batch-size control). cudaDeviceSynchronize() in the CUDA wrapper enforces ordering between launches and future batch-size updates.
- **Current codebase:** Repository contains the naive kernel (kernels/gemm_lib.cu), build via make → lib/libgemm.so, and NVML profiling (profiler.py). Tiled GEMM, cuBLAS, RLC PDN model, and the closed loop are scheduled next

2. Objectives completed (proposal phases)

Phase 1 — Workload and Python–C++ bridge (weeks 1–2)

- **Naive GEMM in CUDA** (gemm_lib.cu), built as a shared object libgemm.so for sm_70.
- **Calling the CUDA code from Python:** We use Python's built-in ctypes module to load the compiled library ('libgemm.so') and invoke 'run_gemm' from the profiler. The script hands over the matrices as float32 NumPy arrays plus the three sizes M, N, and K. In our sweeps we only change M to make each launch more or less work while N and K stay at 1024.
- **Profiler (profiler.py):** background thread samples NVML (pynvml) at 10 ms while the main thread issues repeated kernel launches. Power (W), temperature (°C), utilization (%), SM clock (MHz) are polled.

Phase 2 — Kernel profiling (weeks 3–4)

- Batch-size sweep on V100. $M \in \{64, 128, 256, 512, 1024\}$, fixed $N = K = 1024$ in the driver, 50 launches per configuration.

3. Preliminary results

Data below are from `example_output.txt`, collected on Scholar Tesla V100-PCIE-16GB under that protocol.

batch_size	samples	avg_power_W	max_power_W	avg_util_%	avg_clock_MHz
64	18	36.9	45.6	2.0	930.4
128	9	47.0	47.7	18.0	1245.0
256	10	50.2	54.0	31.0	1245.0
512	16	55.9	59.5	33.1	1245.0
1024	23	60.9	68.9	39.6	1264.3

Observations

- Batch size modulates average power monotonically ($\sim 37\text{ W} \rightarrow \sim 61\text{ W}$), so M is a viable actuator for average power and thus for di/dt seen by a downstream PDN model.
- Peak power reaches 68.9 W at $M = 1024$, about 87% higher than the average at $M = 64$ (36.9 W). A large swing for controller and simulation experiments.
- Utilization stays $\leq \sim 40\%$ even at $M = 1024$, consistent with a naive, bandwidth-bound kernel. Tiled and cuBLAS should raise power and sharpen transients, as intended for the next milestone.
- SM clock jumps from $\sim 930\text{ MHz}$ ($M = 64$) to $\sim 1245\text{ MHz}$ ($M \geq 128$), reflecting boost / residency behavior at low effective load. This will be folded into PDN calibration and analysis (we cannot lock clocks without admin rights on Scholar).

Together, these results validate the measurement stack and show enough power dynamic range to motivate dynamic batch throttling before the RLC and feedback loop are added.

4. Plan for the remainder of the project

Weeks 5–6 (early April) — PDN modeling and richer workloads

- Add shared-memory tiled GEMM and cuBLAS kernels.
- Implement the RLC package PDN in Python, mapping power time series $\rightarrow$ equivalent current $\rightarrow$ simulated Vdroop. Tune R , L , C for plausible droop at peak di/dt .
- Fairer timing for benchmarks: Right now ‘`run_gemm`’ allocates GPU memory at the start of every call and frees it at the end. That extra work is not the matrix multiply itself, but it still takes time, so measured kernel time can be misleading, especially when the real work is short. We plan to allocate A , B , and C on the GPU once and reuse them across many launches so the numbers reflect the actual compute more closely.

Weeks 7–8 (late April) — Control loop and demonstration

- Threshold-based controller: if modeled Vdroop nears Vmin, reduce M. If stable, increase M toward throughput goals, with optional hysteresis to limit chatter.
- FIFO / delay between power samples and PDN state to mimic sensing and RC delay.
- Stacked timelines (power, batch size, Vdroop) for the final report and demo.
- Reporting: GFLOP/s (or time/kernel), and energy metrics.

5. Challenges and mitigations

Challenge	Mitigation
NVML effective resolution (~10 ms) vs. short kernels → sparse or misleading samples at small M	Many repeats per configuration. Accept higher uncertainty at small M. Optionally use CUDA events for true GPU time alongside NVML.
Clock varies with load (e.g., 930 vs 1245 MHz), no root for clock lock on Scholar	Document clock state per sweep, emphasize controller design for $M \geq 128$ where clock is plateaued in our data.
Ordering of kernel launch vs. host batch update	cudaDeviceSynchronize() after launches. Hysteresis in the controller to reduce jitter.
Per-call cudaMalloc / cudaFree in run_gemm	Persistent buffers in the profiling path for fair throughput and cleaner power traces.

6. Success Rubrics

The success of this project will stay the same and be measured against the following engineering criteria:

1. **Workload Launch:** Successful execution of C++ CUDA kernels from a Python host using ctypes bridge without memory segmentation faults.
2. **Batch Size Sensitivity:** Prove that highly optimized GEMM kernels generate di/dt transients at high batch sizes.
3. **Di/dt Mitigation:** The closed-loop controller must show success in preventing simulated voltage drops below Vmin by throttling batch size under peak workload transients.
4. **Throughput Recovery:** The controller must also recover and maximize batch size once the transient noise settles, without generating more transient noise during load step ups.

7. References

- Liang, Zhirui, et al. "GPU-To-Grid: Voltage Regulation via GPU Utilization Control." ArXiv.org, 2026, arxiv.org/abs/2602.05116.